Advanced Statistical Analysis

Assignment 1

경영학과 20230659 유호정

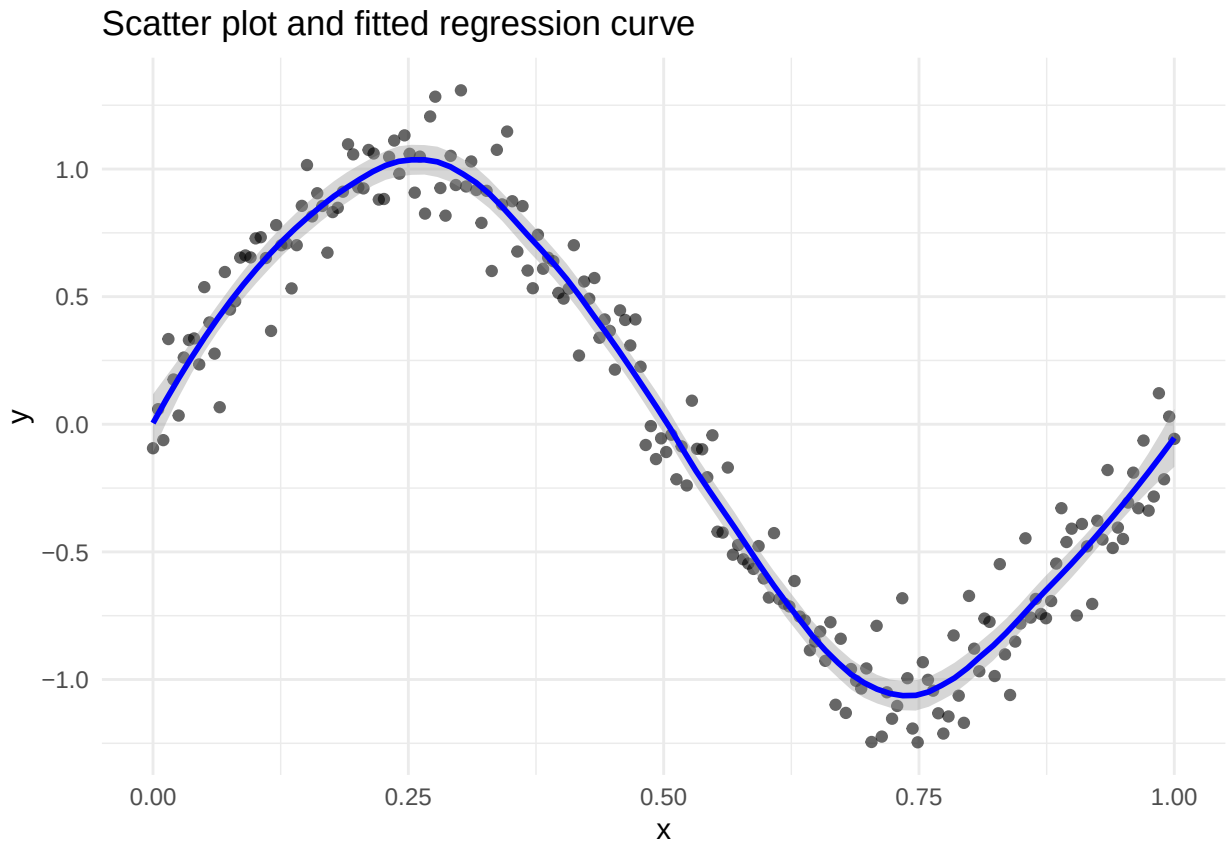
2026-10-13

```r
# Problem 1 ================================================================================


set.seed(1)
n=200
x=seq(0,1,length.out=n)
y=sin(2*pi*x)+rnorm(n,sd=0.15)


# (a)
# save the data as csv in data/ (relative path from the project root)
dir.create("data", showWarnings = FALSE)
dir.create("plots", showWarnings = FALSE)
dir.create("R", showWarnings = FALSE)
write.csv(data.frame(x = x, y = y), "data/p1_data.csv", row.names = FALSE)


# (b)
# write the script file R/p1_plot.R, then run it
# (the true regression function is a sine curve, so loess is used in
#  geom_smooth() instead of a straight line)
script = c(
  "library(ggplot2)",
  "",
  "dat = read.csv('data/p1_data.csv')",
  "",
  "p = ggplot(dat, aes(x = x, y = y)) +",
  "  geom_point(color = 'black', alpha = 0.6) +",
  "  geom_smooth(method = 'loess', formula = y ~ x, span = 0.3,",
  "              color = 'blue', linewidth = 1) +",
  "  labs(title = 'Scatter plot and fitted regression curve',",
  "       x = 'x', y = 'y') +",
  "  theme_minimal()",
  "",
  "print(p)"
)
writeLines(script, "R/p1_plot.R")
source("R/p1_plot.R")
```




```r
# (c)
ggsave("plots/p1_plot.png", plot = p, width = 6, height = 4, dpi = 300)


# (d)
# Commands run in the RStudio Terminal:
#   git add .
#   git commit -m "HW1: project setup, data, plot"
#   git remote add origin "REPO address"
#   git branch -M main
#   git push -u origin main
# GitHub repository link: https://github.com/yooho1928/HW1
```

```r
# Problem 2 ====================================================================


# (a)
# bubble sort: compare neighbors and swap if they are out of order
bubble_sort = function(vec, decreasing = FALSE)
{
  n = length(vec)
  if (n < 2) return(vec)

  for (i in 1:(n-1))
  {
    swapped = FALSE
```

```
    for (j in 1:(n-i))
    {
      if (decreasing == FALSE)
        out_of_order = (vec[j] > vec[j+1])
      else
        out_of_order = (vec[j] < vec[j+1])

      if (out_of_order)
      {
        temp = vec[j]
        vec[j] = vec[j+1]
        vec[j+1] = temp
        swapped = TRUE
      }
    }
    if (swapped == FALSE) break   # early stop: no swap means already sorted
  }
  return(vec)
}

set.seed(1)
x = runif(10)
x
```

```
##  [1] 0.26550866 0.37212390 0.57285336 0.90820779 0.20168193 0.89838968
##  [7] 0.94467527 0.66079779 0.62911404 0.06178627
```

```
bubble_sort(x)                          # ascending
```

```
##  [1] 0.06178627 0.20168193 0.26550866 0.37212390 0.57285336 0.62911404
##  [7] 0.66079779 0.89838968 0.90820779 0.94467527
```

```
bubble_sort(x, decreasing = TRUE)      # descending
```

```
##  [1] 0.94467527 0.90820779 0.89838968 0.66079779 0.62911404 0.57285336
##  [7] 0.37212390 0.26550866 0.20168193 0.06178627
```

```
# compare with the built-in sort()
all.equal(bubble_sort(x), sort(x))
```

```
## [1] TRUE
```

```
all.equal(bubble_sort(x, decreasing = TRUE), sort(x, decreasing = TRUE))
```

```
## [1] TRUE
```

```
# (b)
# quick sort: pivot + recursion on the left and right parts
quick_sort = function(vec, decreasing = FALSE)
```

```
{
  n = length(vec)
  if (n <= 1) return(vec)           # stop recursion when length is 0 or 1

  pivot = vec[1]
  rest = vec[-1]

  if (decreasing == FALSE)
  {
    left = rest[rest < pivot]
    right = rest[rest >= pivot]
  }
  else
  {
    left = rest[rest > pivot]
    right = rest[rest <= pivot]
  }

  return(c(quick_sort(left, decreasing), pivot,
           quick_sort(right, decreasing)))
}

set.seed(1)
x = runif(10)

quick_sort(x)                         # ascending
```

```
##  [1] 0.06178627 0.20168193 0.26550866 0.37212390 0.57285336 0.62911404
##  [7] 0.66079779 0.89838968 0.90820779 0.94467527
```

```
quick_sort(x, decreasing = TRUE)      # descending
```

```
##  [1] 0.94467527 0.90820779 0.89838968 0.66079779 0.62911404 0.57285336
##  [7] 0.37212390 0.26550866 0.20168193 0.06178627
```

```
all.equal(quick_sort(x), sort(x))
```

```
## [1] TRUE
```

```
all.equal(quick_sort(x, decreasing = TRUE), sort(x, decreasing = TRUE))
```

```
## [1] TRUE
```

```
# Problem 3 ====================================================================


# (a)
numerical_diff = function(f, x, h = 1e-6, method = "central")
{
  if (method == "forward")
```

```r
    return((f(x + h) - f(x)) / h)
  else if (method == "backward")
    return((f(x) - f(x - h)) / h)
  else if (method == "central")
    return((f(x + h) - f(x - h)) / (2 * h))
  else
    stop("method must be one of forward, backward, central")
}

# f(x) = cos(x) - x,  analytic derivative f'(x) = -sin(x) - 1
f = function(x)
{
  return(cos(x) - x)
}
f_prime = function(x)
{
  return(-sin(x) - 1)
}

x_grid = seq(0, 2*pi, length.out = 100)

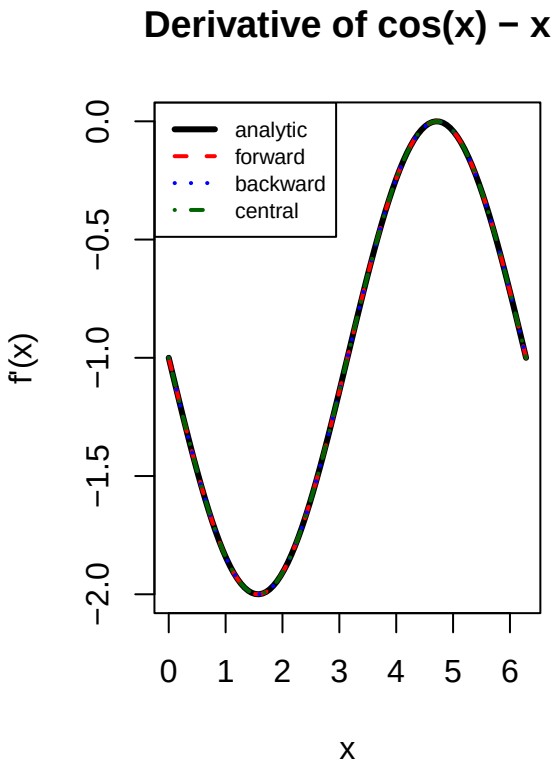
d_true = f_prime(x_grid)
d_forward = numerical_diff(f, x_grid, method = "forward")
d_backward = numerical_diff(f, x_grid, method = "backward")
d_central = numerical_diff(f, x_grid, method = "central")

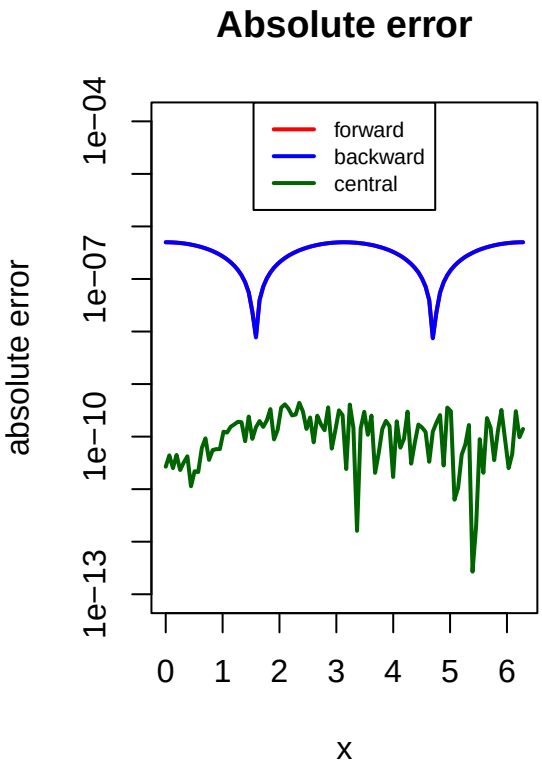
par(mfrow = c(1, 2))

# derivative: analytic vs numerical
plot(x_grid, d_true, type = "l", lwd = 3, xlab = "x", ylab = "f'(x)",
     main = "Derivative of cos(x) - x")
lines(x_grid, d_forward, col = "red", lty = 2, lwd = 2)
lines(x_grid, d_backward, col = "blue", lty = 3, lwd = 2)
lines(x_grid, d_central, col = "darkgreen", lty = 4, lwd = 2)
legend("topleft", c("analytic", "forward", "backward", "central"),
       col = c("black", "red", "blue", "darkgreen"),
       lty = c(1, 2, 3, 4), lwd = c(3, 2, 2, 2), cex = 0.7)

# absolute error against the analytic derivative (log scale)
err_forward = pmax(abs(d_forward - d_true), 1e-16)
err_backward = pmax(abs(d_backward - d_true), 1e-16)
err_central = pmax(abs(d_central - d_true), 1e-16)

matplot(x_grid, cbind(err_forward, err_backward, err_central),
        type = "l", lty = 1, lwd = 2, log = "y", ylim = c(1e-13, 1e-4),
        col = c("red", "blue", "darkgreen"),
        xlab = "x", ylab = "absolute error", main = "Absolute error")
legend("top", c("forward", "backward", "central"),
       col = c("red", "blue", "darkgreen"), lty = 1, lwd = 2, cex = 0.7)
```






```
par(mfrow = c(1, 1))

# maximum absolute error of each method
c(forward = max(abs(d_forward - d_true)),
  backward = max(abs(d_backward - d_true)),
  central = max(abs(d_central - d_true)))
```

```
##      forward     backward      central
## 5.001842e-07 5.001267e-07 4.393228e-10
```

```
# (b)
# Newton-Raphson: if fprime is NULL, use central difference with step h
newton_raphson = function(f, fprime = NULL, x0, maxiter = 100,
                          h = 1e-6, epsilon = 1e-10)
{
  x_old = x0

  for (iter in 1:maxiter)
  {
    if (is.null(fprime))
      slope = numerical_diff(f, x_old, h = h, method = "central")
    else
      slope = fprime(x_old)

    if (slope == 0)
```

```
      stop("derivative is 0, Newton-Raphson cannot continue")

    x_new = x_old - f(x_old) / slope

    if (abs(x_new - x_old) < epsilon)   # convergence check
      return(list(root = x_new, iter = iter, converged = TRUE))

    x_old = x_new
  }

  warning("did not converge within maxiter")
  return(list(root = x_old, iter = maxiter, converged = FALSE))
}


# (c)
# numerical derivative version (fprime = NULL)
res_num = newton_raphson(f, x0 = 0.5)
# analytic derivative version
res_ana = newton_raphson(f, fprime = f_prime, x0 = 0.5)

res_num
```

```
## $root
## [1] 0.7390851
##
## $iter
## [1] 5
##
## $converged
## [1] TRUE
```

```
res_ana
```

```
## $root
## [1] 0.7390851
##
## $iter
## [1] 5
##
## $converged
## [1] TRUE
```

```
print(c(numerical = res_num$root, analytic = res_ana$root), digits = 12)
```

```
##      numerical       analytic
## 0.739085133215 0.739085133215
```

```
f(res_num$root)
```

```
## [1] 0
```

```r
f(res_ana$root)
```

```
## [1] 0
```

```r
# Problem 4 ================================================================================


# (a)
# left rectangle: h * sum_{i=0}^{n-1} f_i
left_rectangle = function(f, a, b, n)
{
  h = (b - a) / n
  x = a + (0:(n-1)) * h
  return(h * sum(f(x)))
}


# (b)
# trapezoid: h/2 * (f_0 + 2 * sum_{i=1}^{n-1} f_i + f_n)
trapezoid = function(f, a, b, n)
{
  h = (b - a) / n
  x = a + (0:n) * h
  fx = f(x)                             # fx[i+1] = f_i
  return(h / 2 * (fx[1] + 2 * sum(fx[-c(1, n+1)]) + fx[n+1]))
}


# (c)
# simpson (n even): h/3 * (f_0 + 4 * sum_odd f_i + 2 * sum_even f_i + f_n)
# the even sum is taken over 0 < i < n, since f_0 and f_n are added separately
simpson = function(f, a, b, n)
{
  if (n %% 2 != 0)
    stop("n must be even for Simpson's rule")

  h = (b - a) / n
  x = a + (0:n) * h
  fx = f(x)                             # fx[i+1] = f_i

  i = 1:(n-1)
  odd_i = i[i %% 2 == 1]
  even_i = i[i %% 2 == 0]

  return(h / 3 * (fx[1] + 4 * sum(fx[odd_i + 1]) +
                    2 * sum(fx[even_i + 1]) + fx[n+1]))
}


# (d)
# integral of sin(x) over [0, pi]; the exact value is 2
n = 100
res_left = left_rectangle(sin, 0, pi, n)
```

```r
res_trap = trapezoid(sin, 0, pi, n)
res_simp = simpson(sin, 0, pi, n)

print(c(left = res_left, trapezoid = res_trap, simpson = res_simp),
      digits = 12)
```

```
##          left     trapezoid       simpson
## 1.99983550389 1.99983550389 2.00000001082
```

```r
# (e)
true_value = 2
n_vec = c(10, 30, 60, 100, 150, 200)

diff_left = numeric(length(n_vec))
diff_trap = numeric(length(n_vec))
diff_simp = numeric(length(n_vec))

for (k in 1:length(n_vec))
{
  diff_left[k] = left_rectangle(sin, 0, pi, n_vec[k]) - true_value
  diff_trap[k] = trapezoid(sin, 0, pi, n_vec[k]) - true_value
  diff_simp[k] = simpson(sin, 0, pi, n_vec[k]) - true_value
}

result = data.frame(n = n_vec, left = diff_left,
                    trapezoid = diff_trap, simpson = diff_simp)
print(result, digits = 3)
```

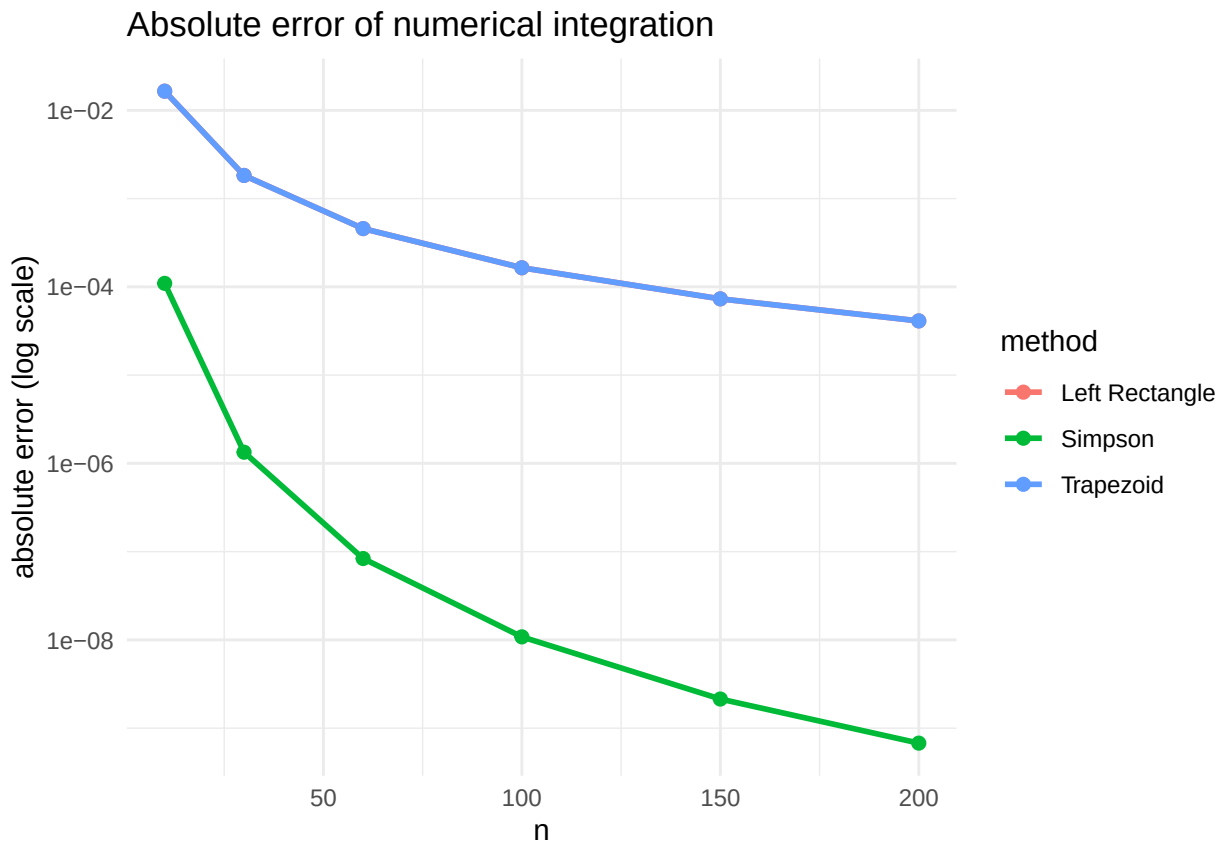
```
##     n      left trapezoid  simpson
## 1  10 -1.65e-02 -1.65e-02 1.10e-04
## 2  30 -1.83e-03 -1.83e-03 1.34e-06
## 3  60 -4.57e-04 -4.57e-04 8.35e-08
## 4 100 -1.64e-04 -1.64e-04 1.08e-08
## 5 150 -7.31e-05 -7.31e-05 2.14e-09
## 6 200 -4.11e-05 -4.11e-05 6.76e-10
```

```r
# absolute error plot (log scale)
# note: sin(0) = sin(pi) = 0, so f_0 = f_n = 0 and the left rectangle
# and trapezoid values coincide (the two lines overlap in the plot)
library(ggplot2)

plot_data = data.frame(
  n = rep(n_vec, 3),
  abs_error = abs(c(diff_left, diff_trap, diff_simp)),
  method = rep(c("Left Rectangle", "Trapezoid", "Simpson"),
               each = length(n_vec))
)

ggplot(plot_data, aes(x = n, y = abs_error, color = method)) +
  geom_line(linewidth = 1) +
  geom_point(size = 2) +
  scale_y_log10() +
```

```
  labs(title = "Absolute error of numerical integration",
       x = "n", y = "absolute error (log scale)") +
  theme_minimal()
```




```
# Problem 5 ================================================================================


# (a)
A = matrix(c(4,2,2,2,5,1,2,1,3), 3)
A
```

```
##      [,1] [,2] [,3]
## [1,]    4    2    2
## [2,]    2    5    1
## [3,]    2    1    3
```

```
U = chol(A)           # A = U^T U, U is upper triangular
L = t(U)              # lower triangular L
L
```

```
##      [,1] [,2]     [,3]
## [1,]    2    0 0.000000
## [2,]    1    2 0.000000
## [3,]    1    0 1.414214
```

```r
L %*% t(L)           # should be equal to A
```

```
##      [,1] [,2] [,3]
## [1,]    4    2    2
## [2,]    2    5    1
## [3,]    2    1    3
```

```r
all.equal(L %*% t(L), A)
```

```
## [1] TRUE
```

```r
# (b)
# forward substitution: solve L z = b
forward = function(L, b)
{
  n = length(b)
  z = numeric(n)

  for (i in 1:n)
  {
    if (i > 1)
      sum_lz = sum(L[i, 1:(i-1)] * z[1:(i-1)])
    else
      sum_lz = 0
    z[i] = (b[i] - sum_lz) / L[i, i]
  }
  return(z)
}

b = c(1, -2, 3)
z = forward(L, b)
z
```

```
## [1]  0.500000 -1.250000  1.767767
```

```r
forwardsolve(L, b)                # compare with the built-in function
```

```
## [1]  0.500000 -1.250000  1.767767
```

```r
all.equal(z, forwardsolve(L, b))
```

```
## [1] TRUE
```

```r
# (c)
# backward substitution: solve L^T x = z
backward = function(L, z)
{
  n = length(z)
  x = numeric(n)
```

```
  for (i in n:1)
  {
    if (i < n)
      sum_lx = sum(L[(i+1):n, i] * x[(i+1):n])    # l_ji, j = i+1,...,n
    else
      sum_lx = 0
    x[i] = (z[i] - sum_lx) / L[i, i]
  }
  return(x)
}

x_back = backward(L, z)
x_back
```

```
## [1] -0.0625 -0.6250  1.2500
```

```
backsolve(t(L), z)                  # t(L) is upper triangular
```

```
## [1] -0.0625 -0.6250  1.2500
```

```
all.equal(x_back, backsolve(t(L), z))
```

```
## [1] TRUE
```

```
# (d)
b = c(1, -2, 3)
z = forward(L, b)
x_sol = backward(L, z)

x_sol
```

```
## [1] -0.0625 -0.6250  1.2500
```

```
solve(A, b)
```

```
## [1] -0.0625 -0.6250  1.2500
```

```
all.equal(x_sol, solve(A, b))
```

```
## [1] TRUE
```

```
# residual A x - b
A %*% x_sol - b
```

```
##               [,1]
## [1,]  0.000000e+00
## [2,]  0.000000e+00
## [3,] -8.881784e-16
```

```r
# Problem 6 ====================================================================


# (a)
# Gaussian kernel: K(x, x') = exp(-rho * ||x - x'||^2)
gaussian_kernel = function(x, x_prime, rho = 1)
{
  return(exp(-rho * sum((x - x_prime)^2)))
}

# kernel matrix: K[i, j] = K(X1[i, ], X2[j, ])
kernel_matrix = function(X1, X2, rho = 1)
{
  X1 = as.matrix(X1)
  X2 = as.matrix(X2)
  n1 = nrow(X1)
  n2 = nrow(X2)

  K = matrix(0, nrow = n1, ncol = n2)
  for (i in 1:n1)
    for (j in 1:n2)
      K[i, j] = gaussian_kernel(X1[i, ], X2[j, ], rho)
  return(K)
}

gaussian_kernel(0, 1)      # exp(-1)
```

```
## [1] 0.3678794
```

```r
exp(-1)
```

```
## [1] 0.3678794
```

```r
# (b)
# fit KRR: alpha = (K + lambda I)^{-1} y, returned with class "krr"
fit_krr = function(X, y, rho = 1, lambda = 0.0001)
{
  X = as.matrix(X)
  n = nrow(X)

  K = kernel_matrix(X, X, rho)
  alpha = solve(K + lambda * diag(n), y)

  model = list(alpha = alpha, X = X, y = y, rho = rho, lambda = lambda)
  class(model) = "krr"
  return(model)
}


# (c)
# predict method: f_hat(x) = k(x, X)^T alpha
predict.krr = function(object, newdata, ...)
```

```
{
  if (missing(newdata))
    newdata = object$X

  K_new = kernel_matrix(newdata, object$X, object$rho)
  return(as.vector(K_new %*% object$alpha))
}


# (d)
# plot method: scatter plot and fitted function
plot.krr = function(x, ...)
{
  model = x
  grid_x = seq(min(model$X), max(model$X), length.out = 200)
  grid_pred = predict(model, matrix(grid_x, ncol = 1))

  plot(model$X[, 1], model$y, pch = 16, col = "gray50",
       xlab = "x", ylab = "y", ...)
  lines(grid_x, grid_pred, col = "blue", lwd = 2)
}


# (e)
set.seed(1)
n = 150
X = matrix(runif(n, -1, 1), ncol = 1)
ftrue = function(x) sin(2*pi*x) + 0.5*cos(4*pi*x)
y = ftrue(X[,1]) + rnorm(n, sd = 0.1)

fit1 = fit_krr(X, y)                # default: rho = 1, lambda = 0.0001
fit2 = fit_krr(X, y, rho = 5)       # larger rho for comparison
class(fit1)
```

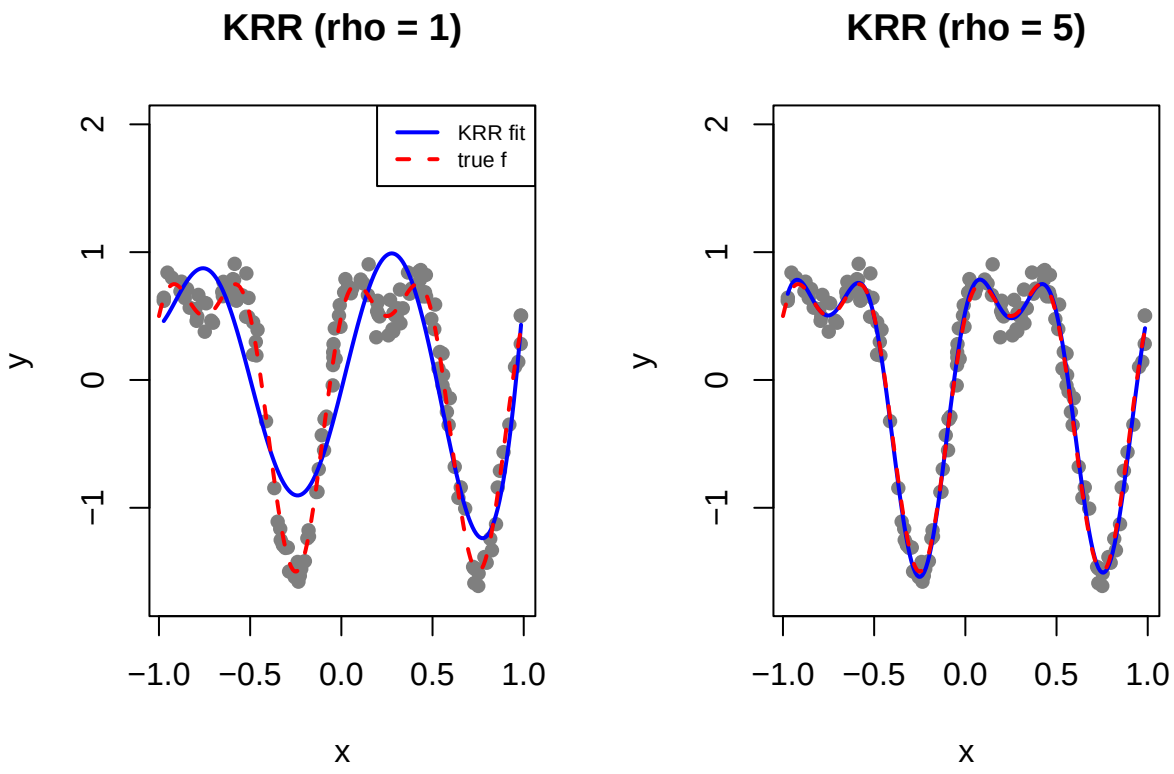
```
## [1] "krr"
```

```
grid_x = seq(-1, 1, length.out = 200)

par(mfrow = c(1, 2))

plot(fit1, main = "KRR (rho = 1)", ylim = c(-1.7, 2))
lines(grid_x, ftrue(grid_x), col = "red", lty = 2, lwd = 2)
legend("topright", c("KRR fit", "true f"), col = c("blue", "red"),
       lty = c(1, 2), lwd = 2, cex = 0.7)

plot(fit2, main = "KRR (rho = 5)", ylim = c(-1.7, 2))
lines(grid_x, ftrue(grid_x), col = "red", lty = 2, lwd = 2)
```




```
par(mfrow = c(1, 1))

# predictions at new points vs the true function
new_x = matrix(c(-0.5, 0, 0.5), ncol = 1)
predict(fit1, new_x)
```

```
## [1]  0.008939831 -0.075240928  0.147330941
```

```
predict(fit2, new_x)
```

```
## [1] 0.4944975 0.5380585 0.5219529
```

```
ftrue(c(-0.5, 0, 0.5))
```

```
## [1] 0.5 0.5 0.5
```

```
# RMSE against the true function on the grid
rmse = function(pred, truth)
{
  return(sqrt(mean((pred - truth)^2)))
}
rmse(predict(fit1, matrix(grid_x, ncol = 1)), ftrue(grid_x))
```

```
## [1] 0.31544
```

```r
rmse(predict(fit2, matrix(grid_x, ncol = 1)), ftrue(grid_x))
```

```
## [1] 0.02732521
```